PHENIKAA UNIVERSITY

Faculty of Computer Science

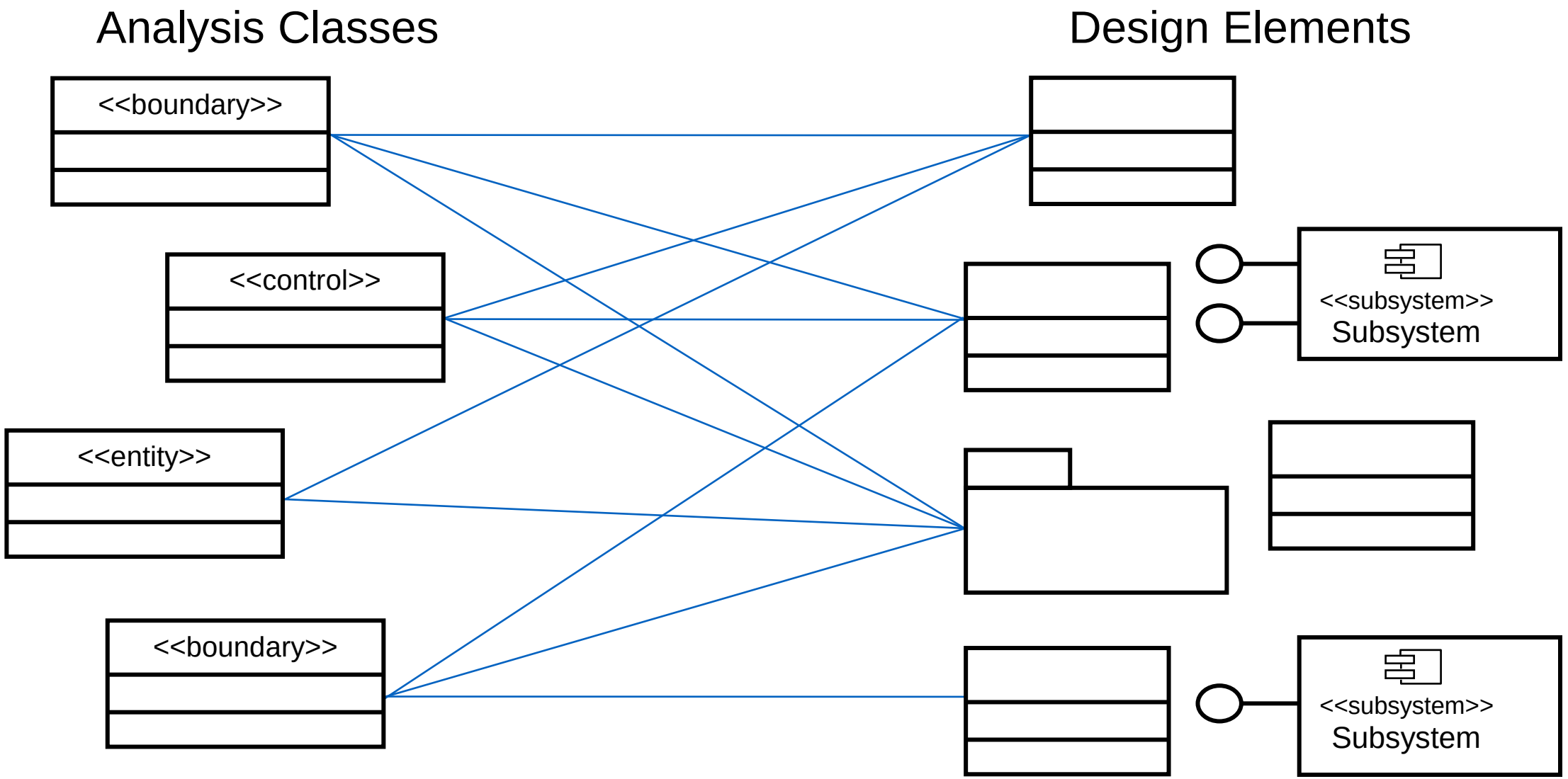
Software Analysis and Design

Module 9: Design Element

Content

- Identify classes and subsystems
- Identify subsystem interfaces
- Identify reuse opportunities
- Update the organization of the Design Model
- Checkpoints

From Analysis Classes to Design Elements



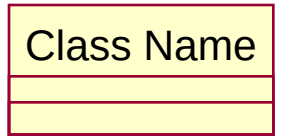
From Analysis Classes to Design Elements

- An analysis class maps directly to a design class if:
  - It is a simple class
  - It represents a single logical abstraction
- More complex analysis classes may
  - Split into multiple classes
  - Become a package
  - Become a subsystem (discussed later)
  - Any combination …

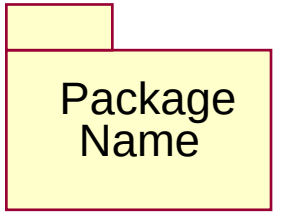
Class and package

PHENIKAA UNIVERSITY
Faculty of Computer Science

- What is a class?
  - A description of a set of objects that share the same responsibilities, relationships, operations, attributes, and semantics



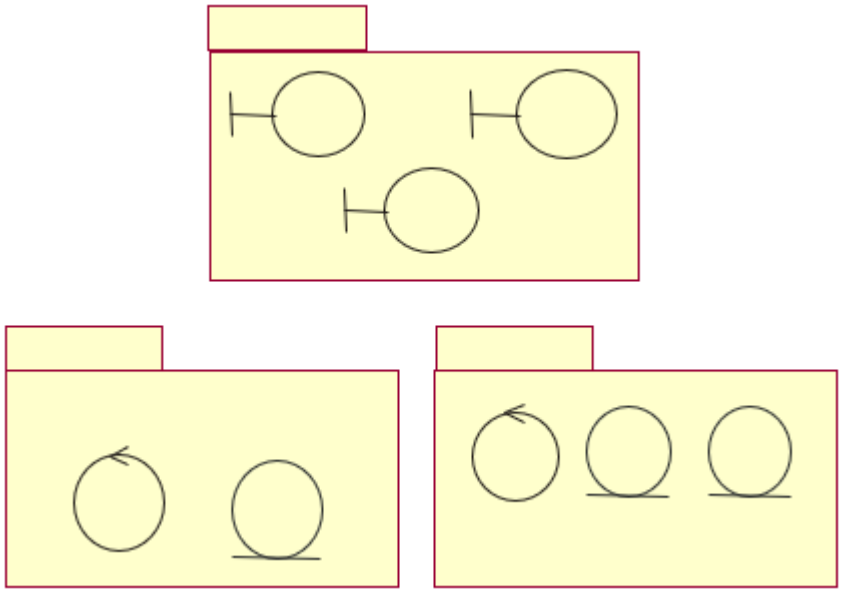
- What is a package?
  - A general-purpose mechanism for organizing elements into groups
  - A model element which can contain



Packaging Tips: Boundary Classes

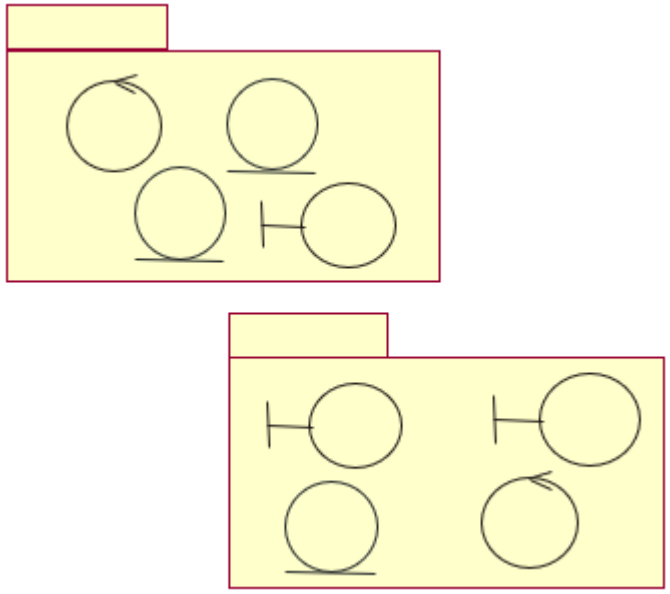
PHENIKAA UNIVERSITY
Faculty of Computer Science

- If it is likely the system interface will undergo considerable changes



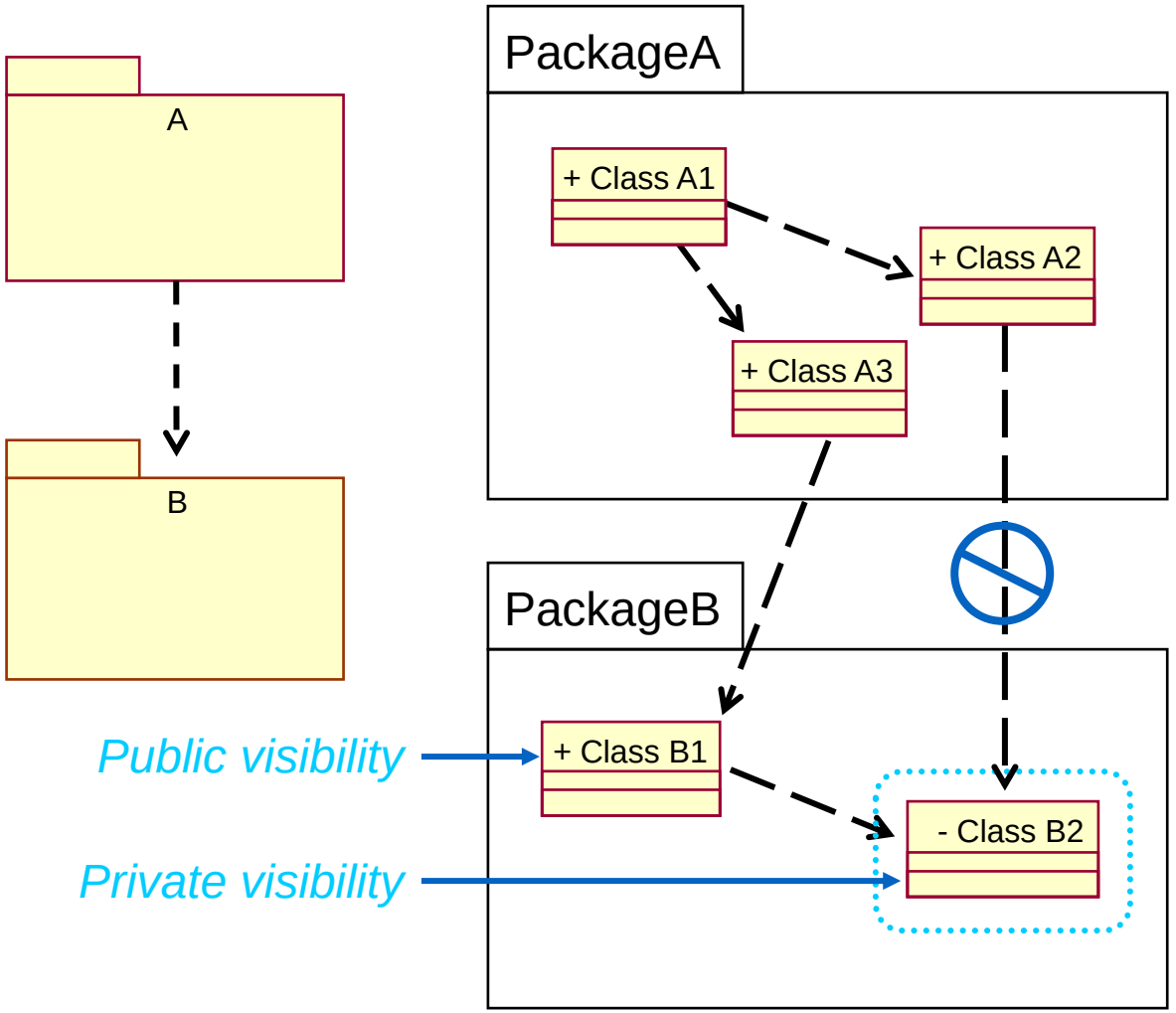
Boundary classes placed in separate packages

- If it is unlikely the system interface will undergo considerable changes



Boundary classes packaged with functionally related classes

Package Dependencies: Package Element Visibility

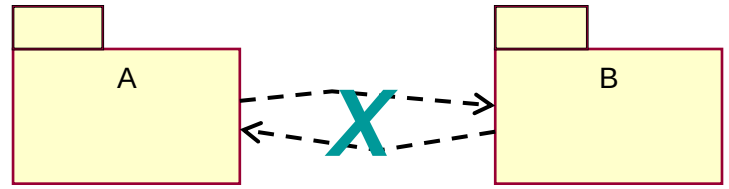


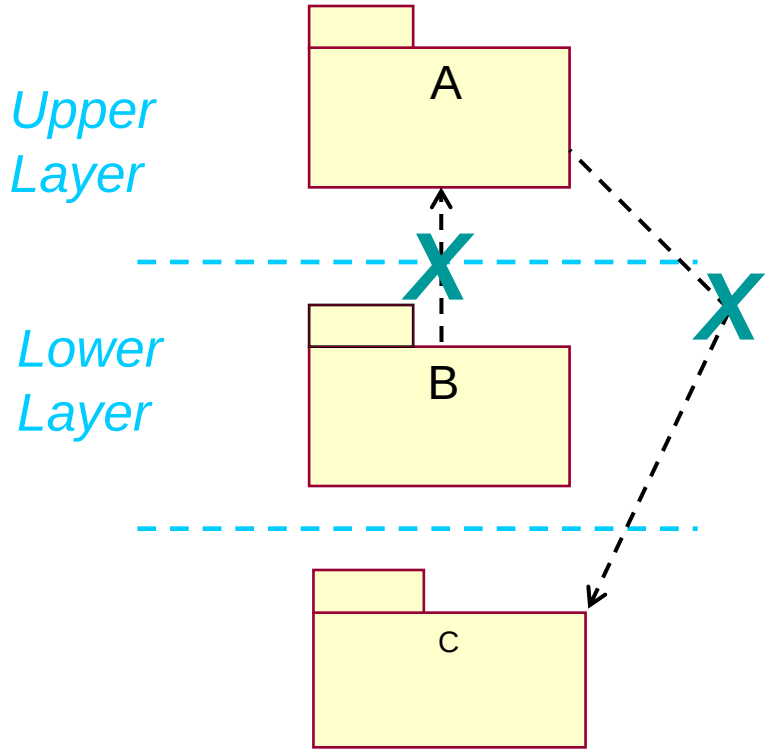
Only public classes can be referenced outside of the owning package

OO Principle: Encapsulation

Package Coupling: Tips

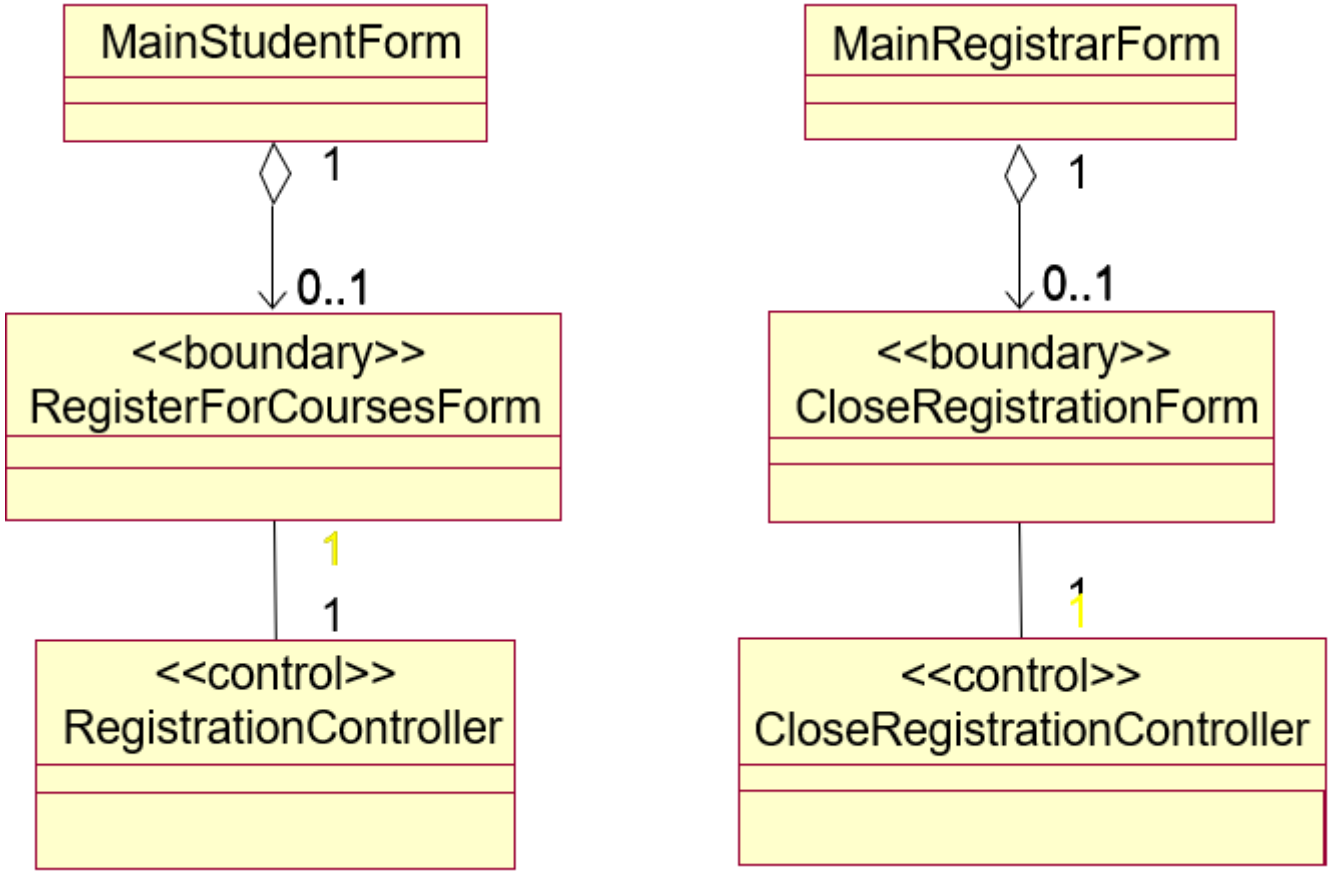
- Packages should not be cross-coupled
- Packages in lower layers should not be dependent upon packages in upper layers
- In general, dependencies should not skip layers



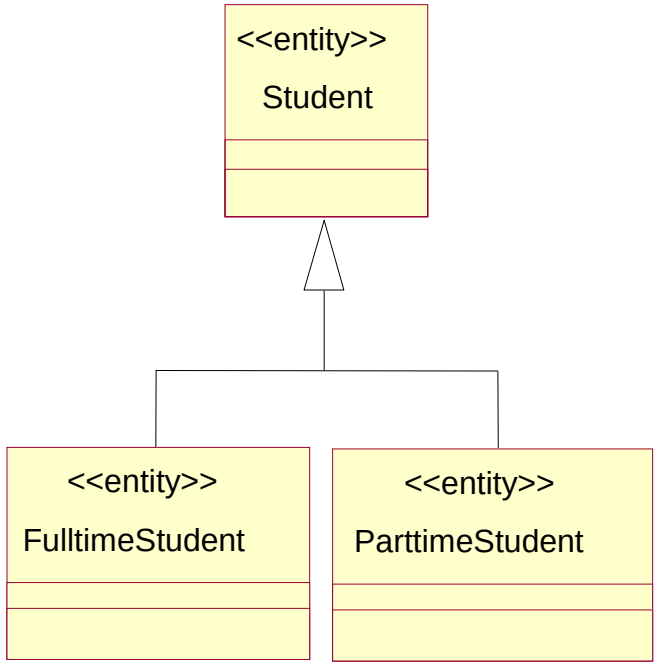


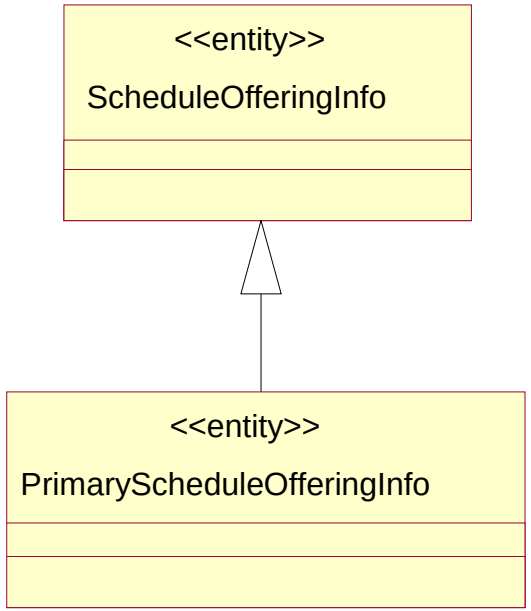
X = Coupling violation

Example: Registration Package

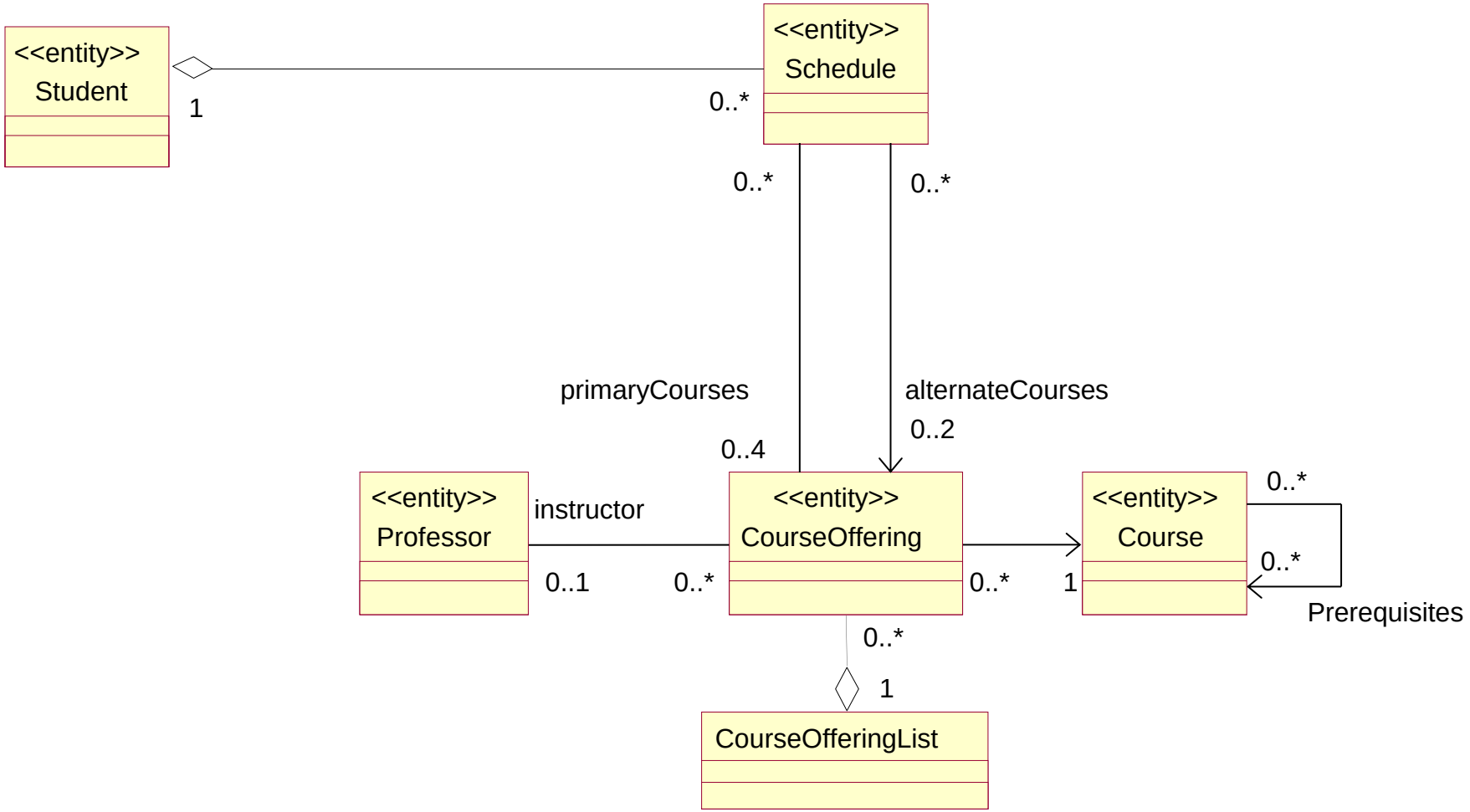


Example: Generalization of University Artifacts Package





Example: Associations of University Artifacts Package

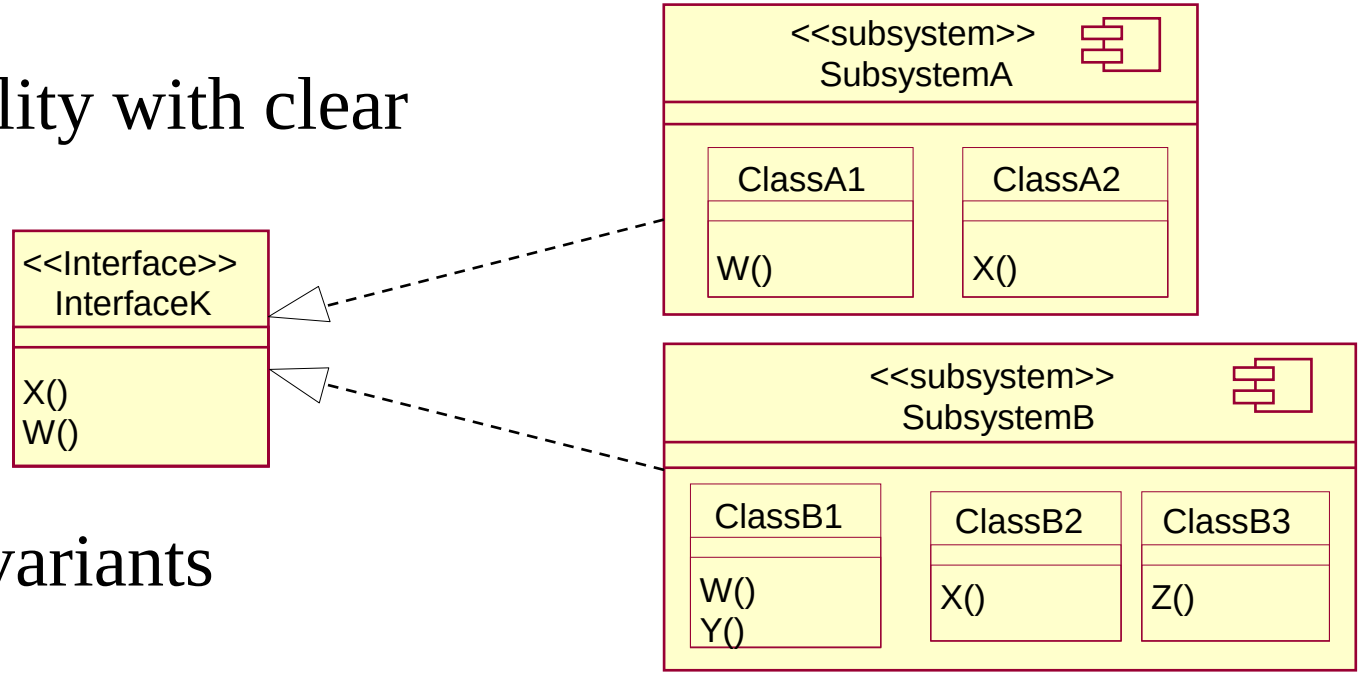


Content

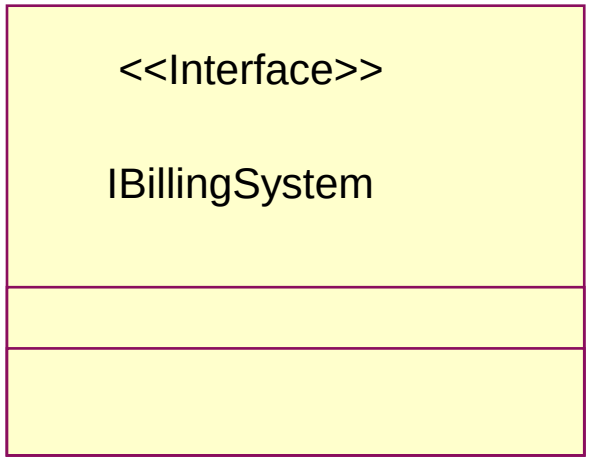
- Identify classes and subsystems
- Identify subsystem interfaces
- Identify reuse opportunities
- Update the organization of the Design Model
- Checkpoints

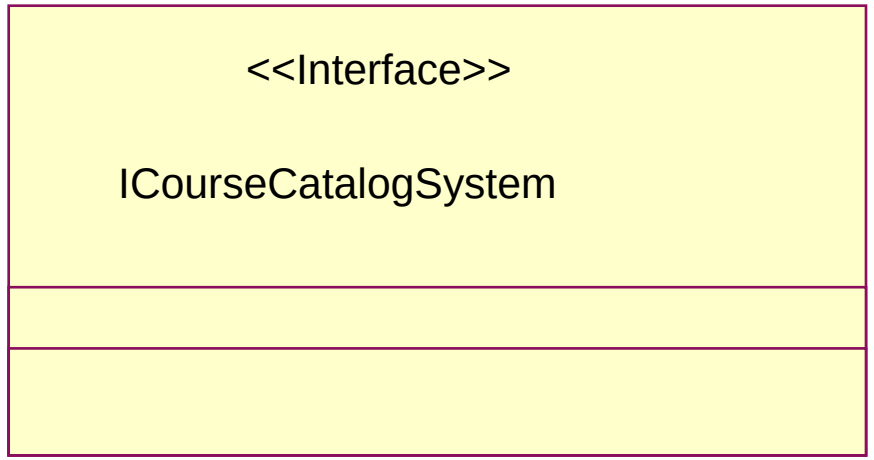
Subsystem

- Completely encapsulate behavior
- Represent an independent capability with clear interfaces (potential for reuse)
- Model multiple implementation variants



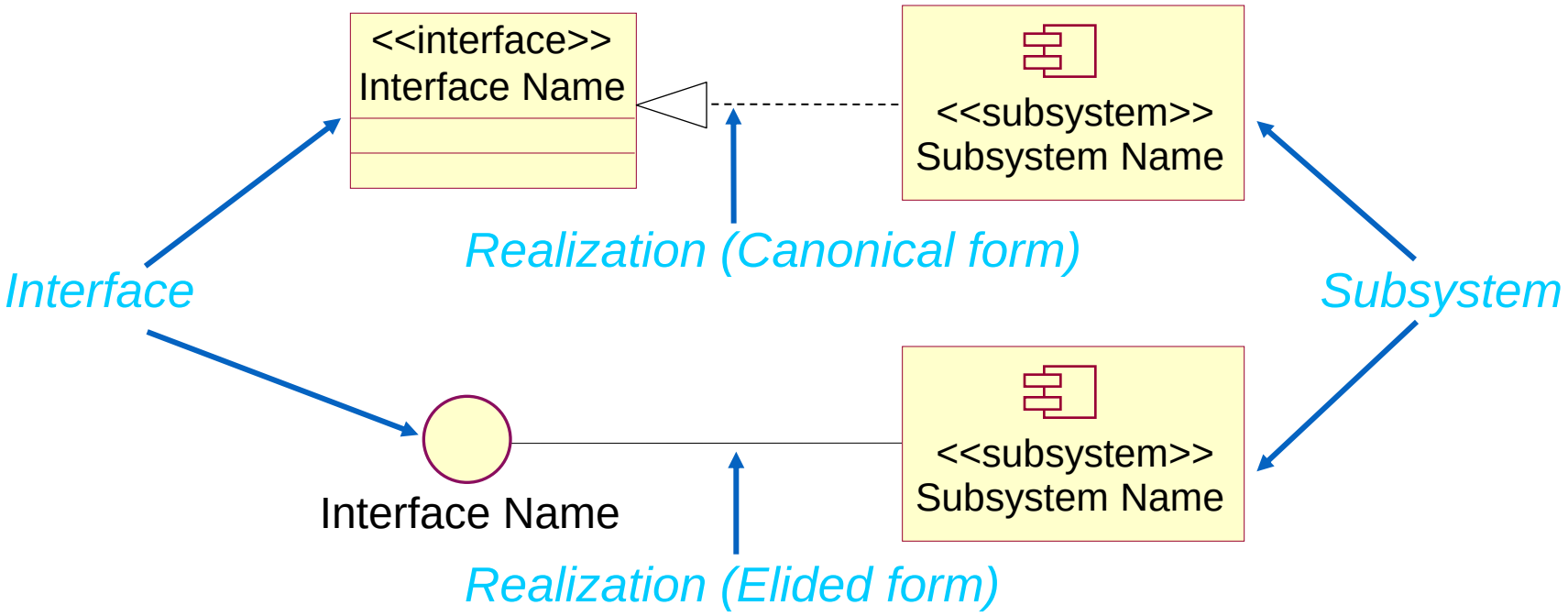
Example: External System Interfaces Package





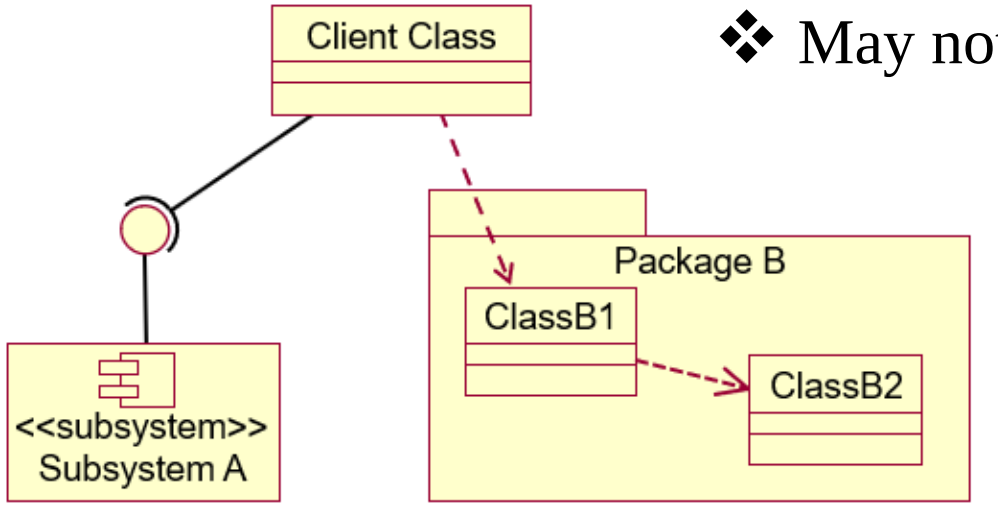
Subsystems and Interfaces

❖ Realizes one or more interfaces that define its behavior



Packages versus Subsystems

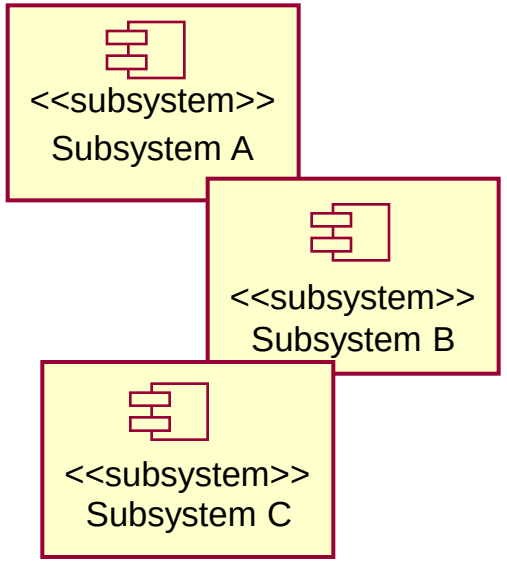
- Subsystems
  - Provide behavior
  - Completely encapsulate their contents
  - Are easily replaced
- Packages
  - Don’t provide behavior
  - Don’t completely encapsulate their contents
  - May not be easily replaced



Encapsulation is the key!

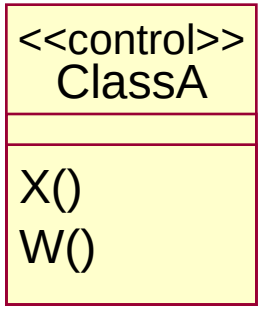
Candidate Subsystems

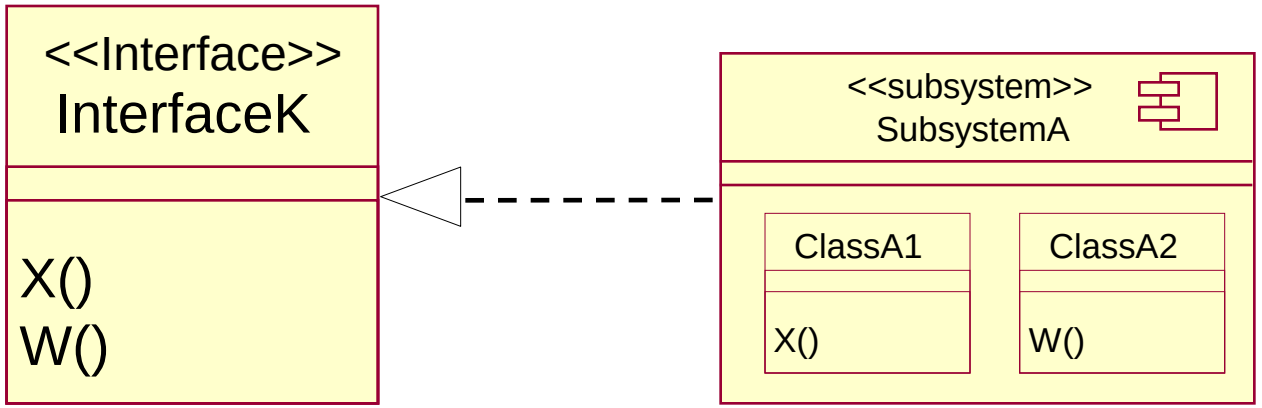
- Analysis classes which may evolve into subsystems:
  - Classes providing complex services and/or utilities
  - Boundary classes (user interfaces and external system interfaces)
- Existing products or external systems in the design (e.g., components):
  - Communication software
  - Database access support
  - Types and data structures
  - Common utilities
  - Application-specific products



Identifying Subsystems

"Superman Class"





Candidate Subsystems

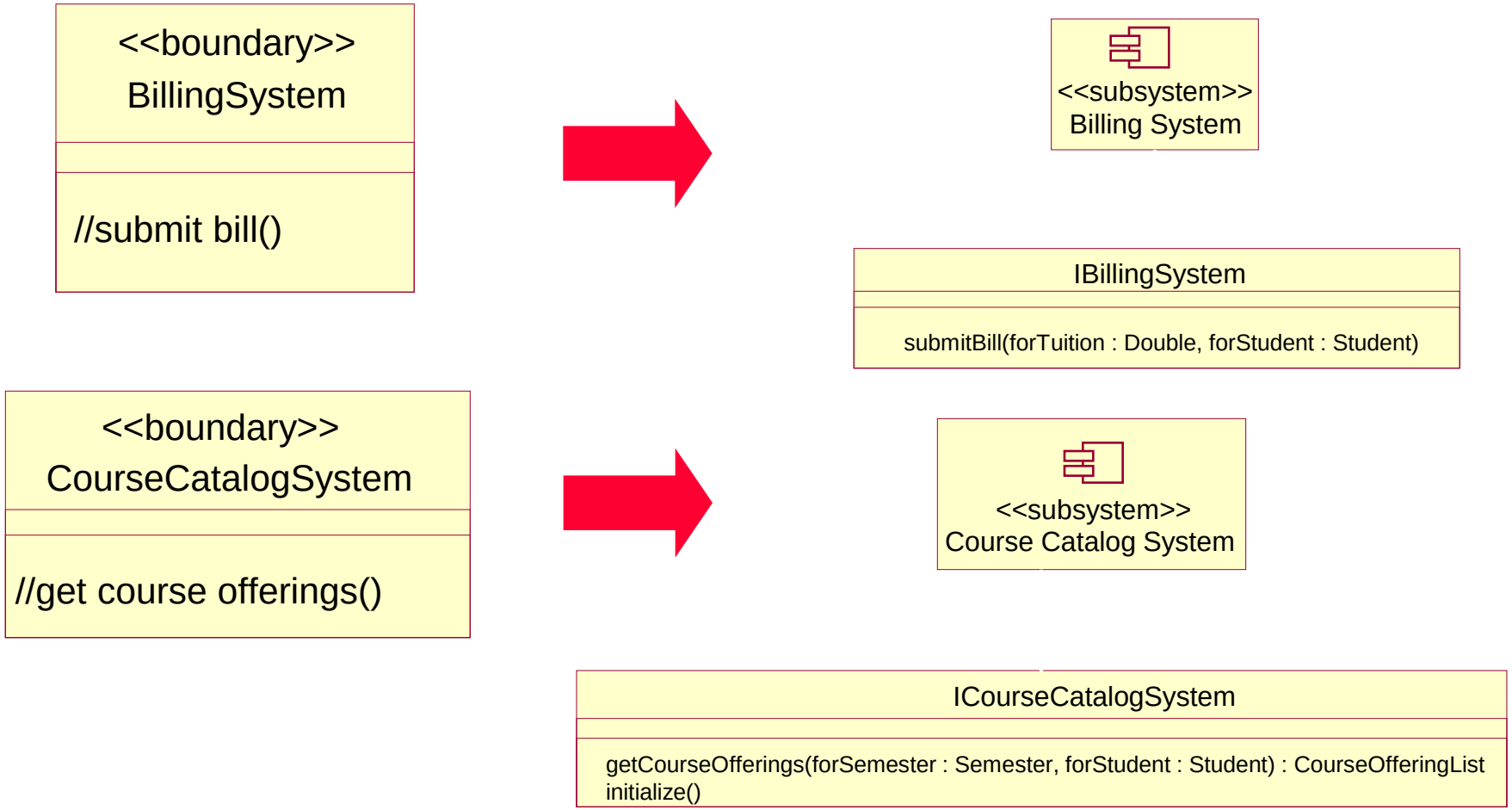
- Purpose
  - To identify the interfaces of the subsystems based on their responsibilities
- Steps
  - Identify a set of candidate interfaces for all subsystems.
  - Look for similarities between interfaces.
  - Define interface dependencies.
  - Map the interfaces to subsystems.
  - Define the behavior specified by the interfaces…
  - Package the interfaces

Stable, well-defined interfaces are key to a stable, resilient architecture.

Interface Guideline

- Interface name
  - Reflects role in system
- Interface description
  - Conveys responsibilities
- Operation definition
  - Name should reflect operation result
  - Describes what operation does, all parameters and result
- Interface documentation
  - Package supporting info: sequence and state diagrams, test plans, etc.

Example: Design Subsystems and Interfaces

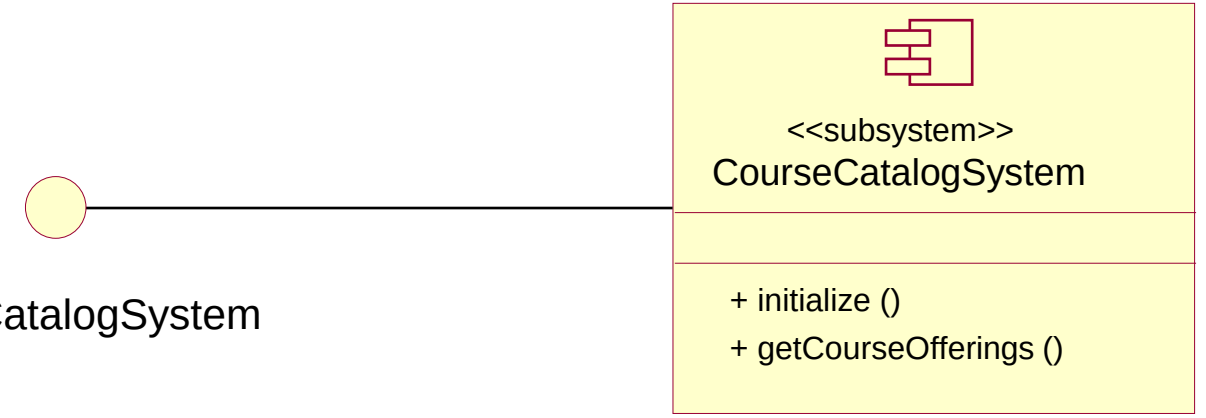


All other analysis classes map directly to design classes.

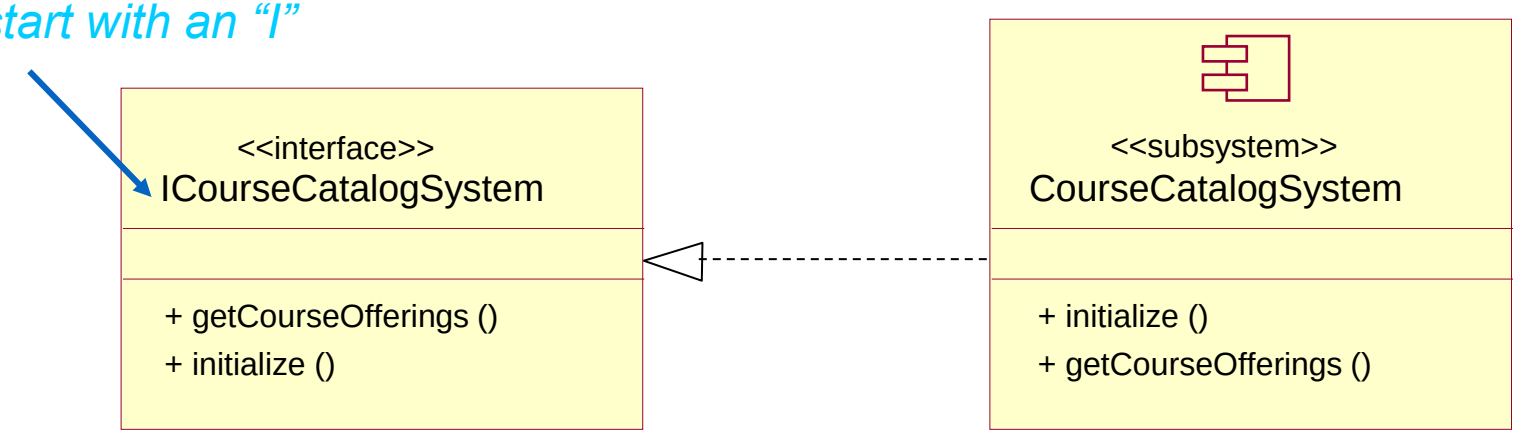
Example: Analysis-Class-To-Design-Element Map

Analysis Class	Design Element
CourseCatalogSystem	CourseCatalogSystem Subsystem
BillingSystem	BillingSystem Subsystem
All other analysis classes map directly to design classes	

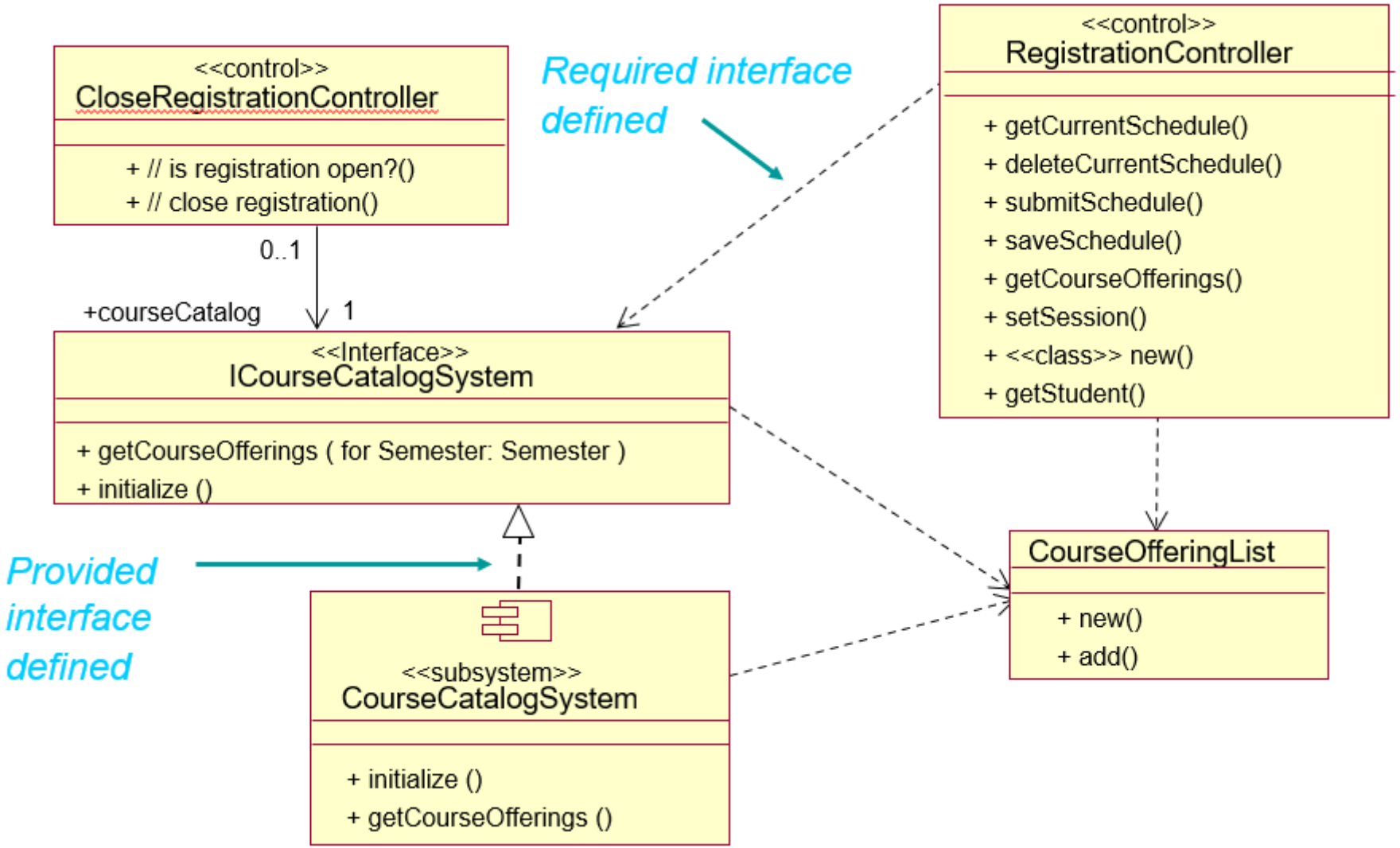
Modeling Convention: Subsystems and Interfaces



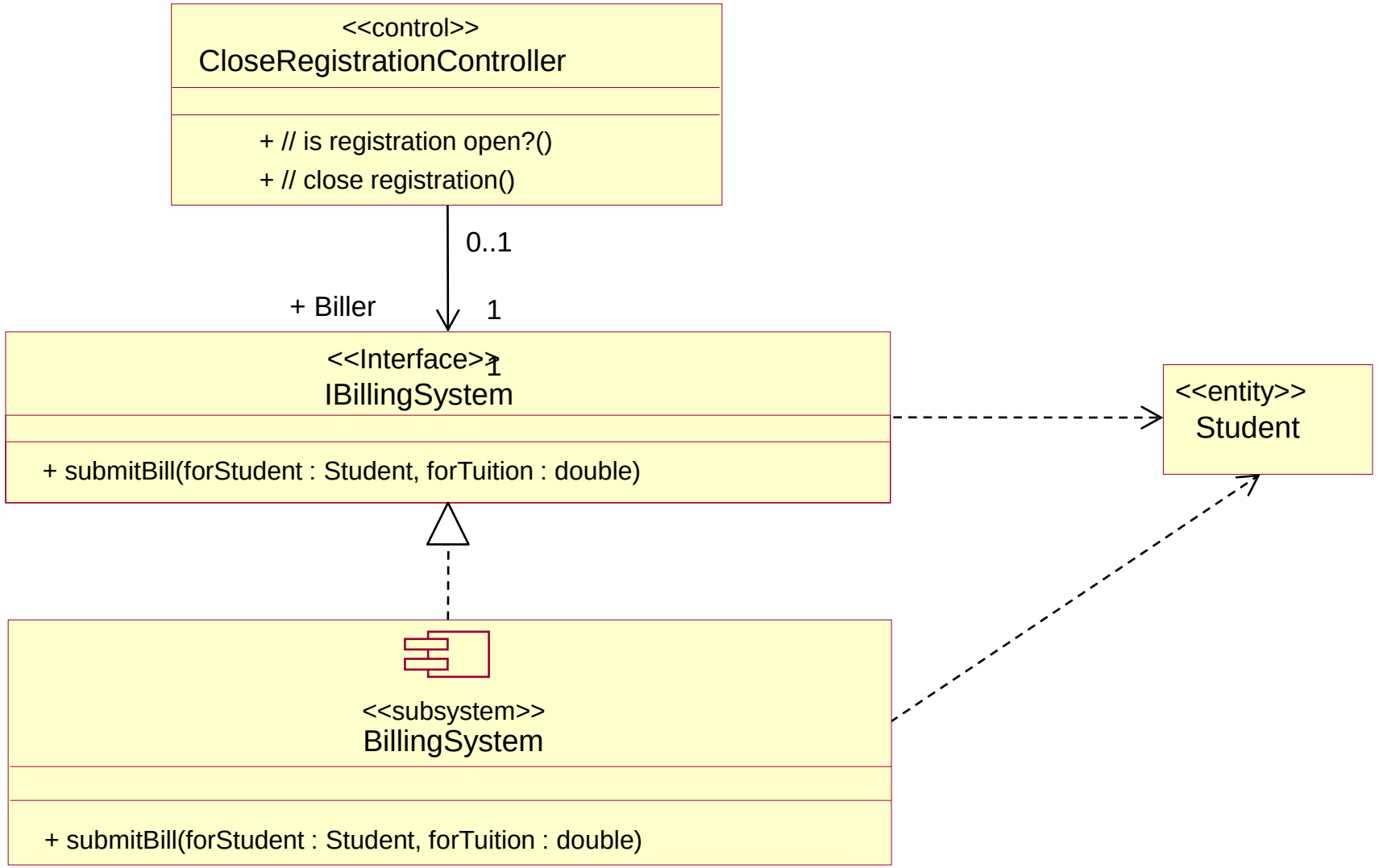
Interfaces start with an "I"



Example: Subsystem Context of CourseCatalogSystem



Example: Subsystem Context of Billing System



Content

- Identify classes and subsystems
- Identify subsystem interfaces
- Identify reuse opportunities
- Update the organization of the Design Model
- Checkpoints

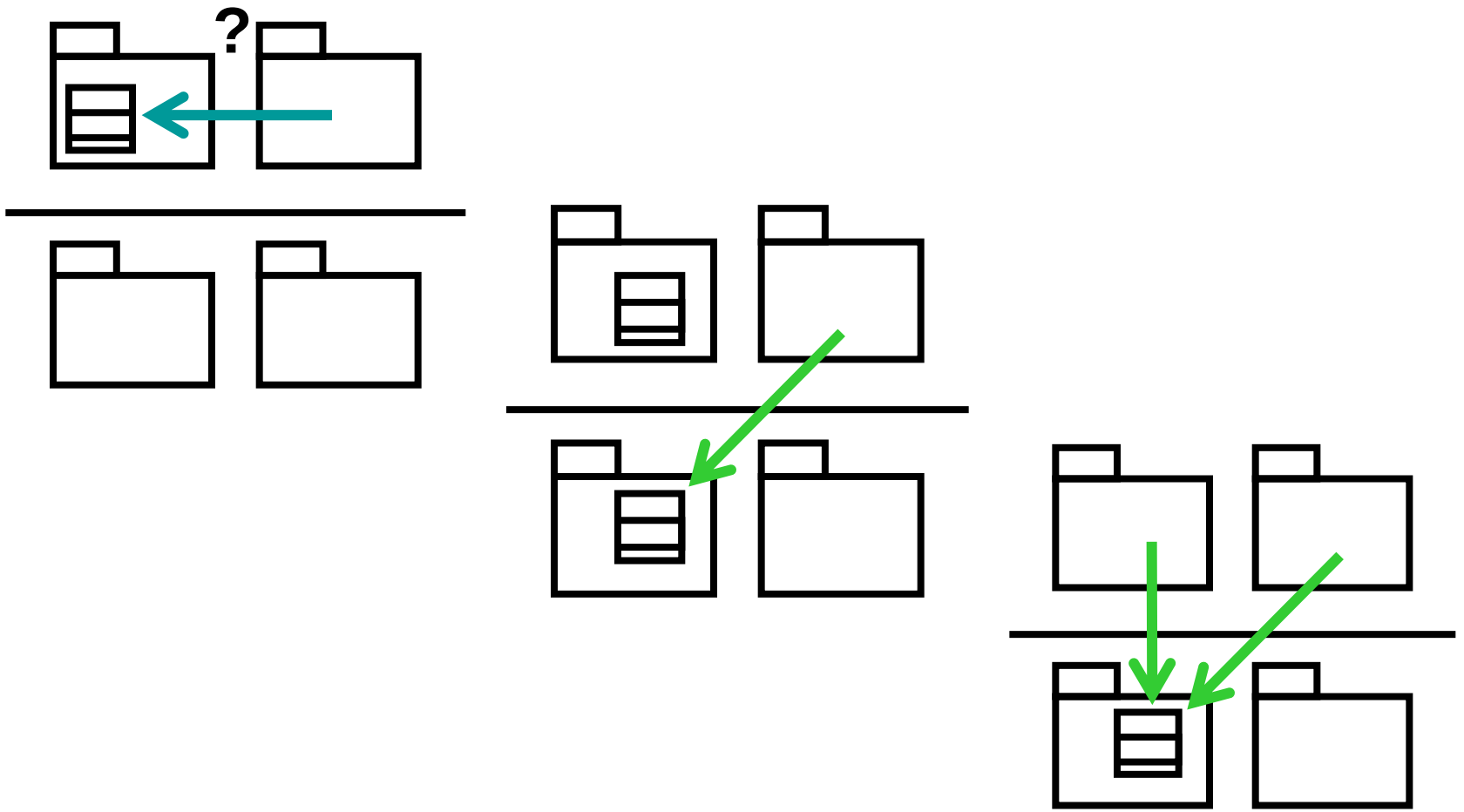
Identification of Reuse Opportunities

- Purpose
  - To identify where existing subsystems and/or components can be reused based on their interfaces.
- Steps
  - Look for similar interfaces
  - Modify new interfaces to improve the fit
  - Replace candidate interfaces with existing interfaces
  - Map the candidate subsystem to existing components

Possible Reuse Opportunities

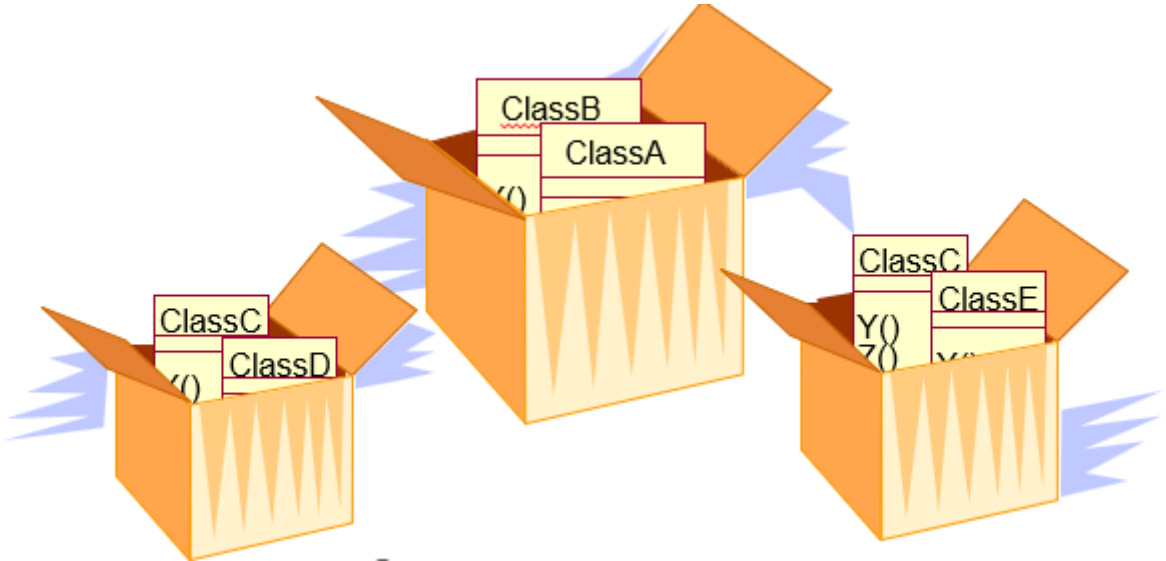
- Internal to the system being developed
  - Recognized commonality across packages and subsystems
- External to the system being developed
  - Commercially available components
  - Components from a previously developed application
  - Reverse engineered components

Reuse Opportunities Internal to System

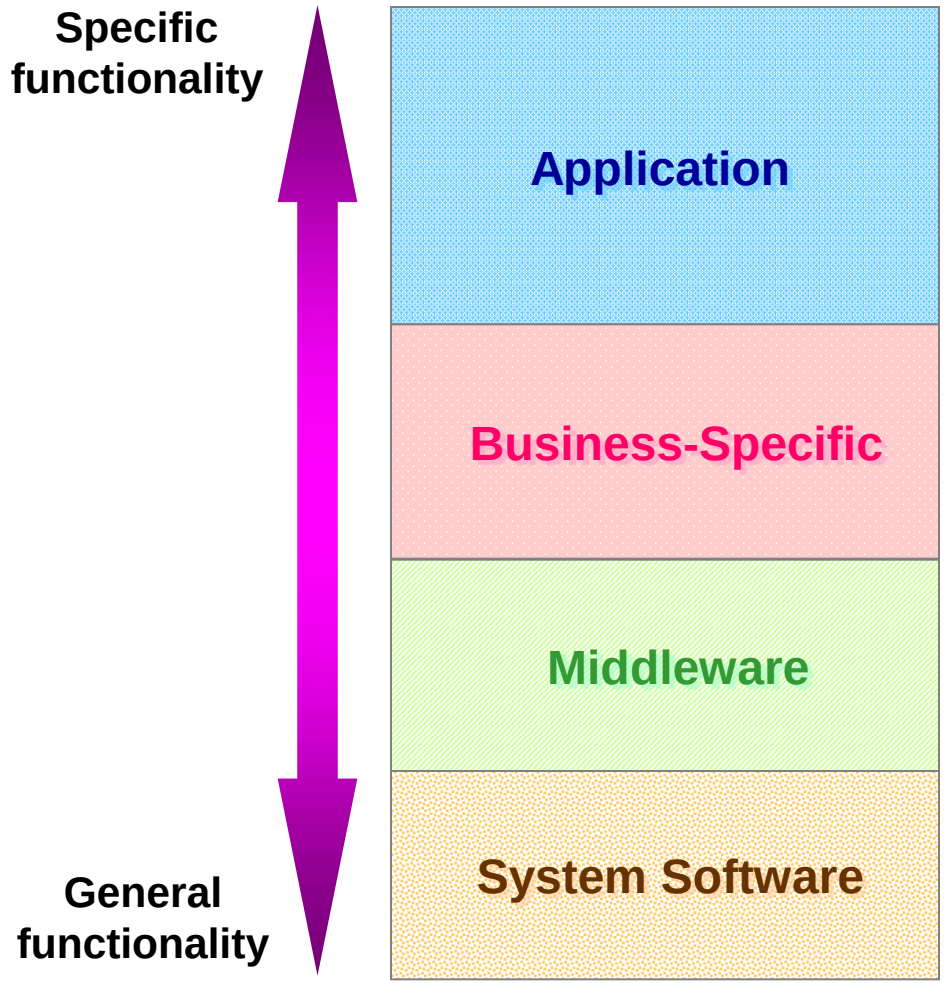


Content

- Identify classes and subsystems
- Identify subsystem interfaces
- Identify reuse opportunities
- Update the organization of the Design Model
- Checkpoints



Typical Layering Approach



Distinct application subsystems that make up an application — contains the value adding software developed by the organization.

Business specific — contains a number of reusable subsystems specific to the type of business.

Middleware — offers subsystems for utility classes and platform-independent services for distributed object computing in heterogeneous environments and so on.

System software — contains the software for the actual infrastructure such as operating systems, interfaces to specific hardware, device drivers, and so on.

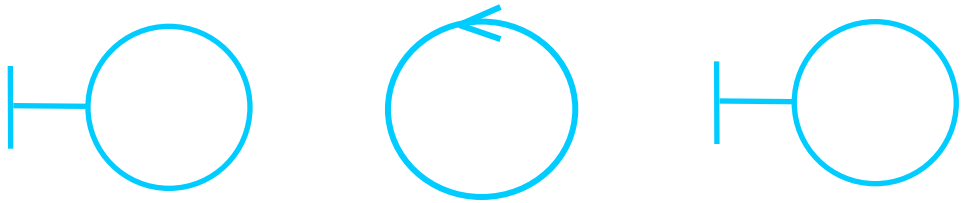
Layering Considerations

- Visibility
  - Dependencies only within current layer and below
- Volatility
  - Upper layers affected by requirements changes
  - Lower layers affected by environment changes
- Generality
  - More abstract model elements in lower layers
- Number of layers
  - Small system: 3-4 layers
  - Complex system: 5-7 layers

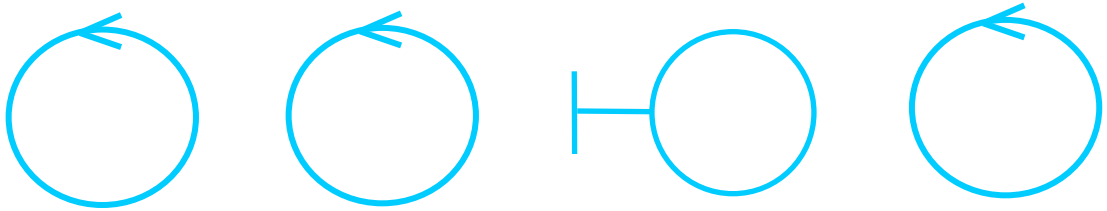
Goal is to reduce coupling and to ease maintenance effort.

Design Elements and the Architecture

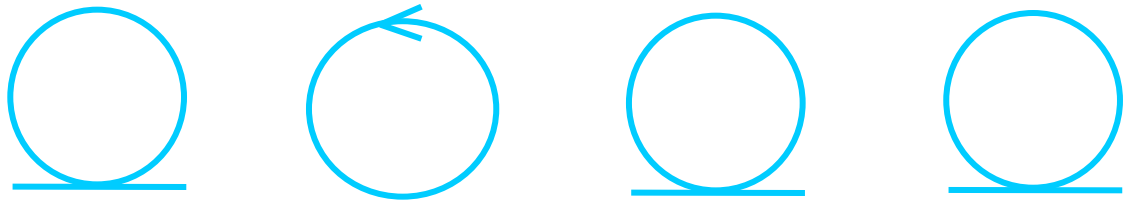
Layer 1



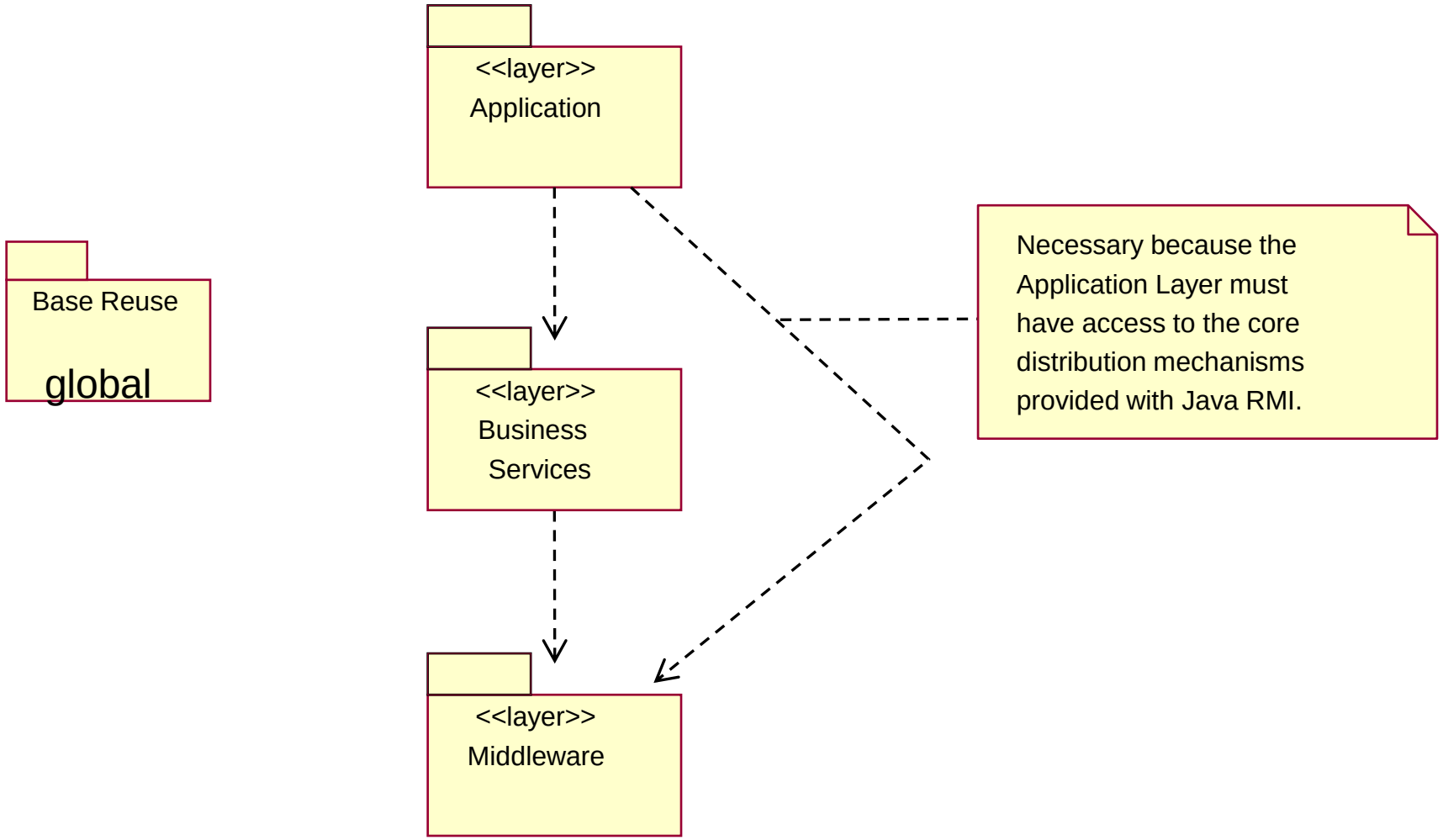
Layer 2



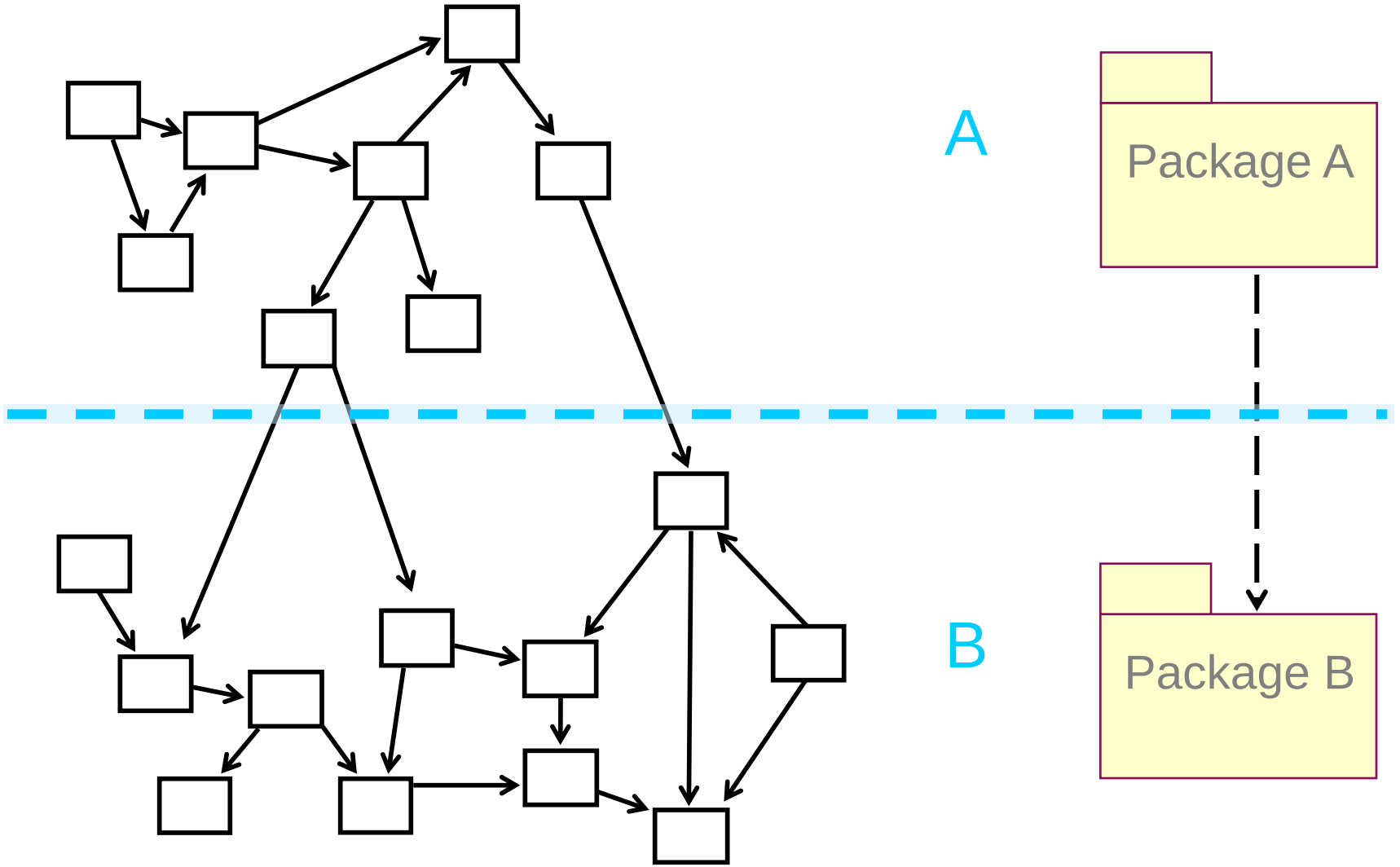
Layer 3



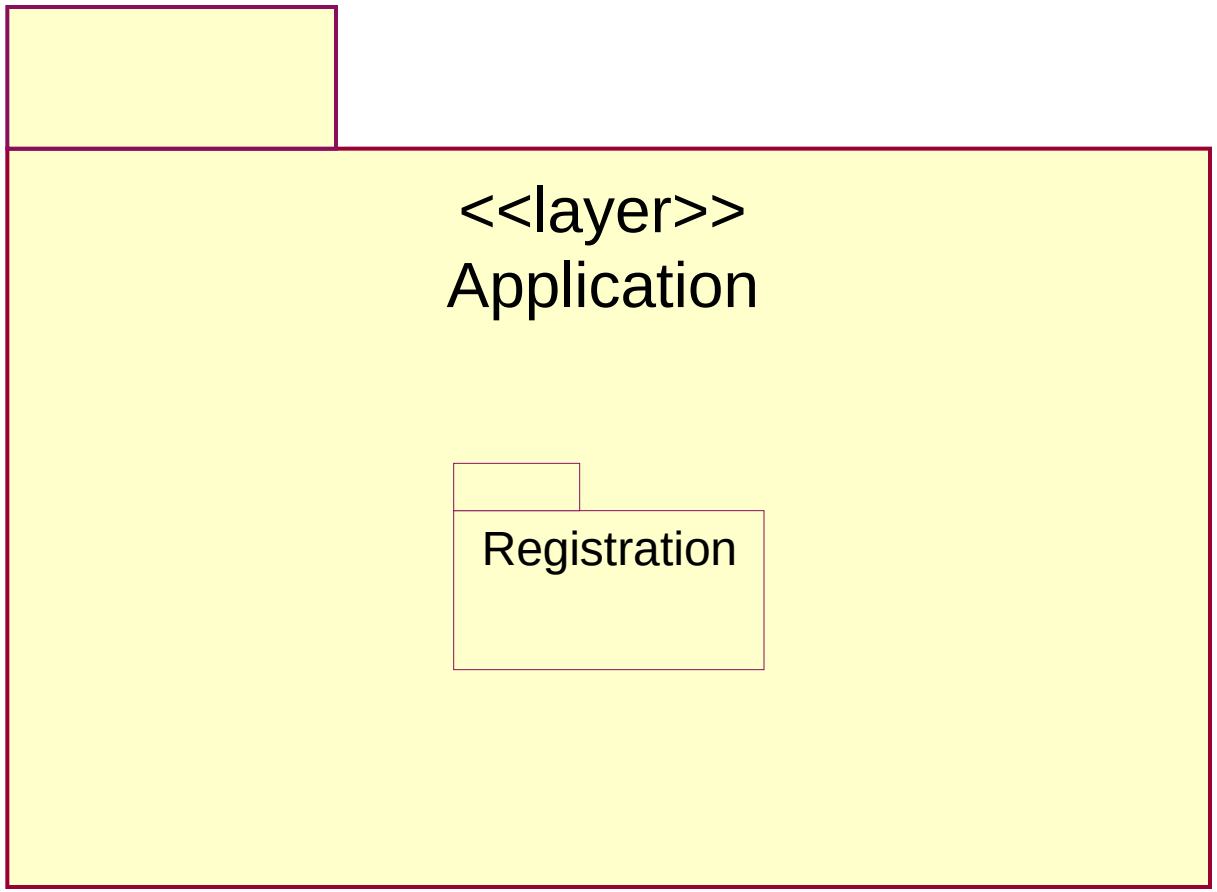
Example: Architectural Layers



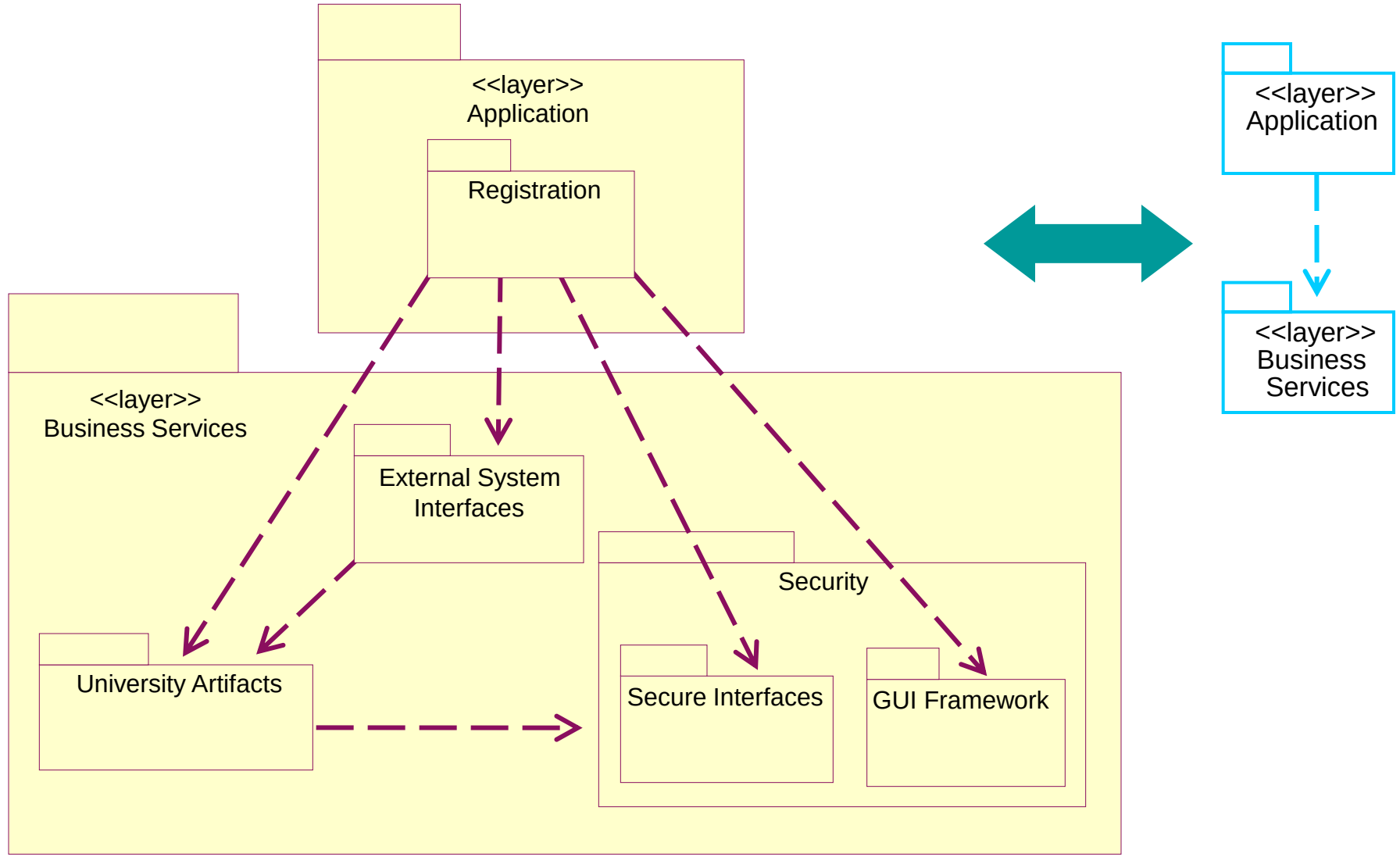
Example: Partitioning



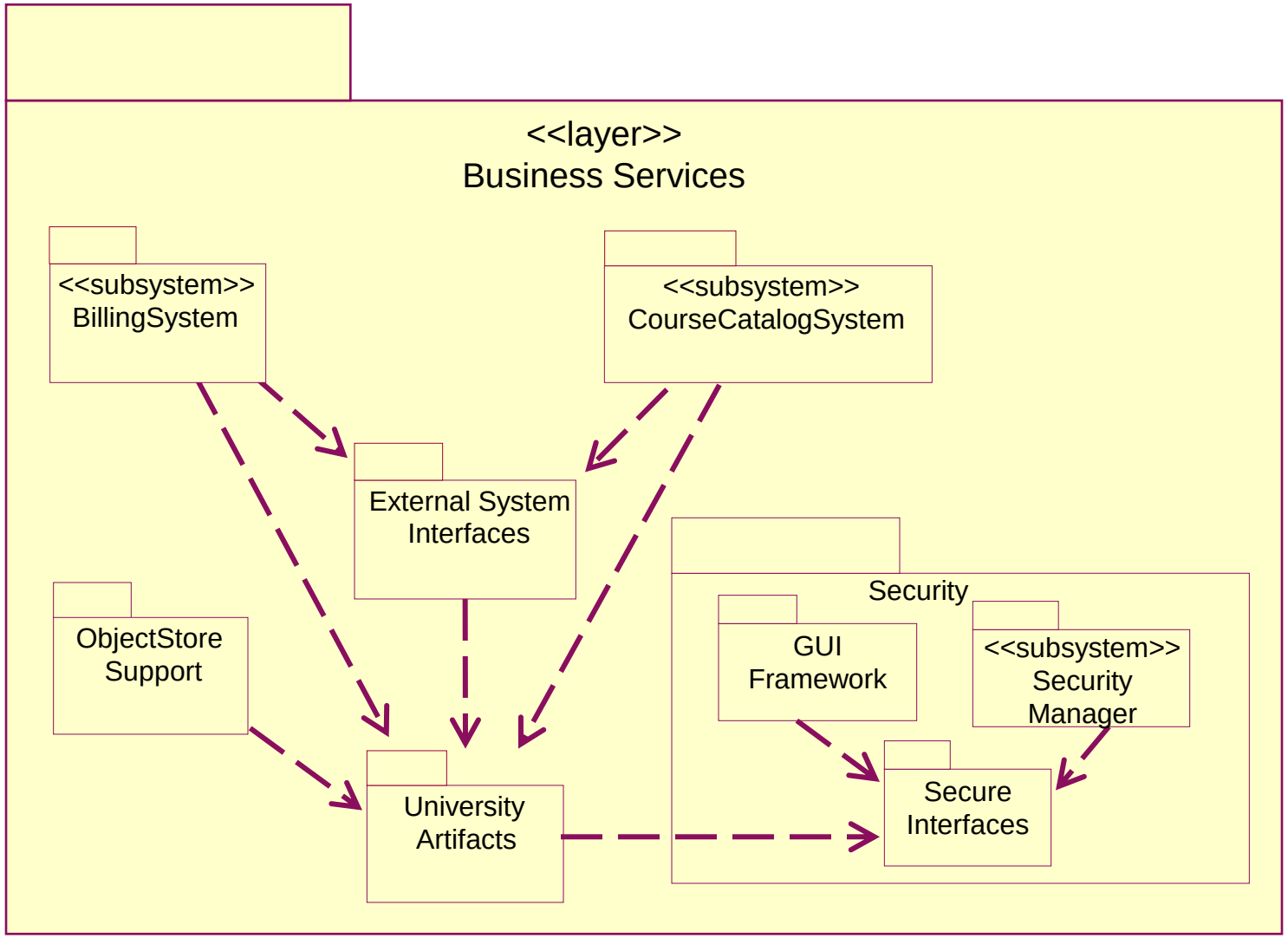
Example: Application Layer



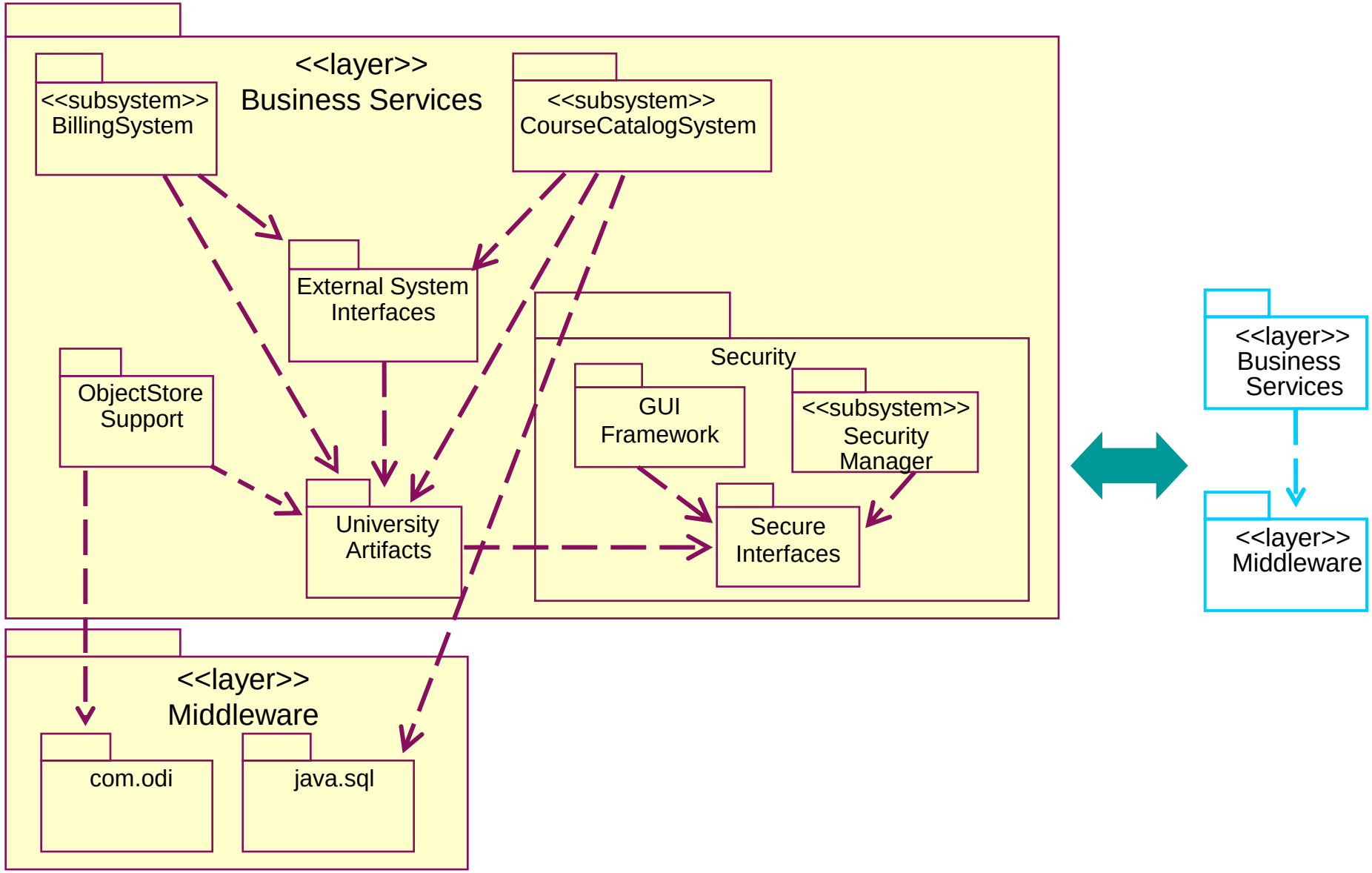
Example: Business Services Layer



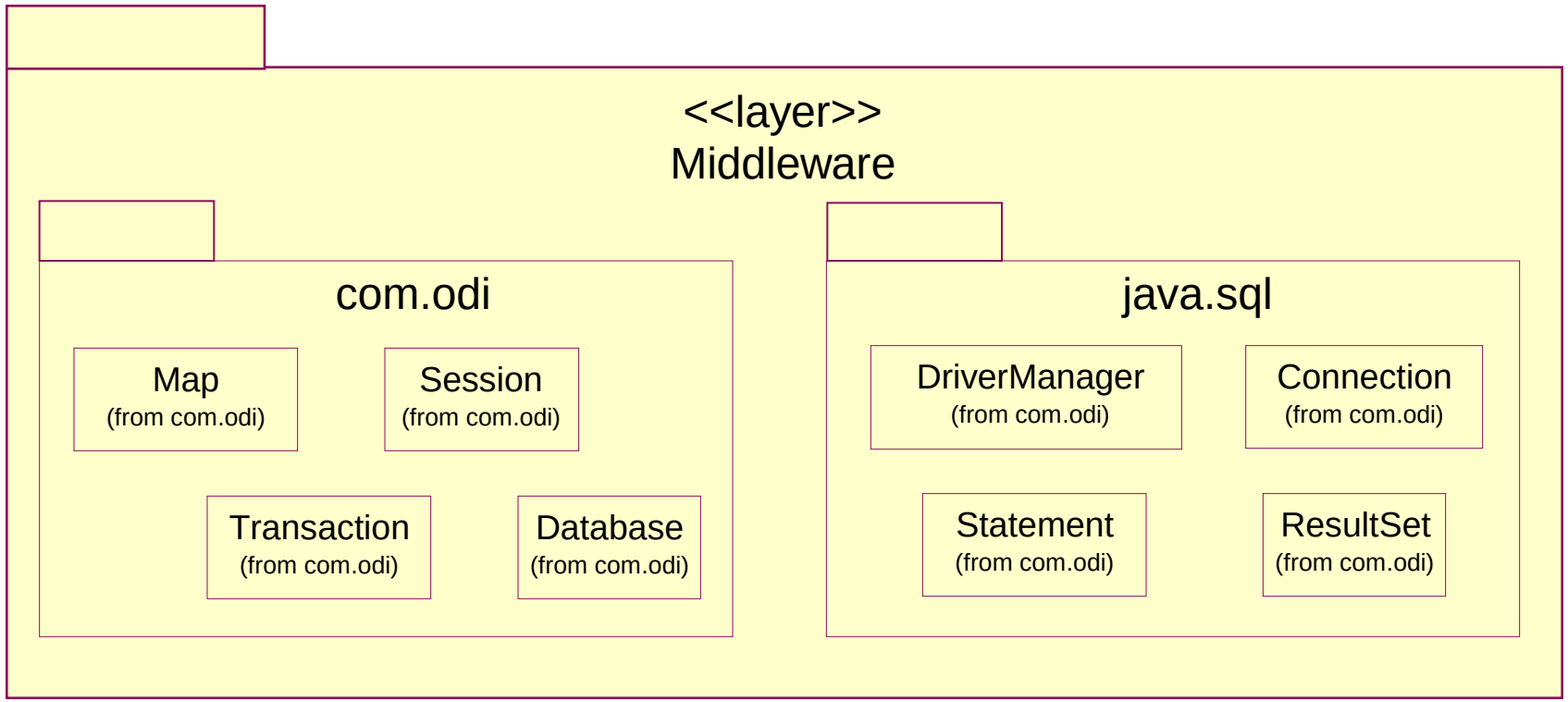
Design Elements and the Architecture



Example: Business Services Layer Context



Example: Middleware Layer



Content

- Identify classes and subsystems
- Identify subsystem interfaces
- Identify reuse opportunities
- Update the organization of the Design Model
- Checkpoints

Checkpoints

- General
  - Does it provide a comprehensive picture of the services of different packages?
  - Can you find similar structural solutions that can be used more widely in the problem domain?
- Layers
  - Are there more than seven layers?
- Subsystems
  - Is subsystem partitioning done in a logically consistent way across the entire model?

Checkpoints: Packages

- Are the names of the packages descriptive?
- Does the package description match with the responsibilities of contained classes?
- Do the package dependencies correspond to the relationships between the contained classes?
- Do the classes contained in a package belong there according to the criteria for the package division?
- Are there classes or collaborations of classes within a package that can be separated into an independent package?
- Is the ratio between the number of packages and the number of classes appropriate?

Checkpoints: Classes

- Does the name of each class clearly reflect the role it plays?
- Is the class cohesive (i.e., are all parts functionally coupled)?
- Are all class elements needed by the use-case realizations?
- Do the role names of the aggregations and associations accurately describe the relationship?
- Are the multiplicities of the relationships correct?

Checkpoints: Classes

- Does the name of each class clearly reflect the role it plays?
- Is the class cohesive (i.e., are all parts functionally coupled)?
- Are all class elements needed by the use-case realizations?
- Do the role names of the aggregations and associations accurately describe the relationship?
- Are the multiplicities of the relationships correct?

Review: Identify Design Elements

- What is the purpose of Identify Design Elements?
- What is an interface?
- What is a subsystem? How does it differ from a package?
- What is a subsystem used for, and how do you identify them?
- What are some layering and partitioning considerations?

PHENIKAA UNIVERSITY

Faculty of Computer Science

Q&A